Nama :Veva Br Tarigan

NIM :253303621280

Kelas :3 Pagi B

Jurusan :Sistem Informasi

Mata Kuliah :Dr.Evta Indra,S.Kom.,M.Kom.

Java Visual

1. Ekosistem Java. Jelaskan perbedaan JVM, JRE, dan JDK dengan analogi sehari-hari. Jelaskan peran bytecode dalam "Write Once, Run Anywhere" dan satu batasannya. Mengapa JDK 17 dipilih, bukan JDK 8?
   Jawaban:
   Java memiliki tiga komponen penting, yaitu JVM (Java Virtual Machine), JRE (Java Runtime Environment), dan JDK (Java Development Kit). Ketiganya memiliki fungsi yang berbeda, tetapi saling berhubungan dalam proses pembuatan dan menjalankan program Java.
   Perbedaan JVM, JRE, dan JDK dapat dijelaskan dengan analogi memasak.

- JVM (Java Virtual Machine) diibaratkan sebagai kompor yang menjalankan proses memasak. Dalam Java, JVM bertugas menjalankan bytecode agar program dapat dieksekusi.
- JRE (Java Runtime Environment) diibaratkan sebagai dapur lengkap dengan kompor dan perlengkapan yang dibutuhkan untuk memasak. JRE menyediakan JVM dan komponen pendukung untuk menjalankan program Java.
- JDK (Java Development Kit) diibaratkan sebagai dapur lengkap yang juga memiliki alat untuk membuat resep baru. JDK menyediakan berbagai alat pengembangan, termasuk compiler untuk mengubah kode sumber Java menjadi bytecode.

  Java memiliki konsep Write Once, Run Anywhere, yang berarti program dapat ditulis satu kali dan dijalankan pada berbagai sistem operasi yang mendukung Java. Hal ini dimungkinkan karena kode sumber Java (.java) dikompilasi menjadi bytecode (.class). Bytecode kemudian dijalankan oleh JVM yang sesuai dengan sistem operasi yang digunakan. Dengan demikian, programmer tidak perlu menulis ulang seluruh program untuk setiap sistem operasi.

  Namun, konsep tersebut memiliki batasan. Program tetap membutuhkan JVM yang kompatibel. Selain itu, jika program menggunakan fitur khusus sistem operasi atau library tertentu yang tidak tersedia pada platform lain, program mungkin memerlukan penyesuaian agar dapat berjalan dengan baik.

  Dalam pembuatan aplikasi Character Manager, JDK 17 dipilih karena merupakan versi LTS (Long-Term Support) yang menyediakan dukungan jangka panjang serta berbagai peningkatan dibandingkan JDK 8. JDK 17 juga mendukung pengembangan aplikasi Java

modern dan sesuai digunakan dalam pembelajaran pemrograman berorientasi objek. Sementara itu, JDK 8 merupakan versi yang lebih lama dan tidak menyediakan seluruh fitur yang tersedia pada Java versi lebih baru. Meskipun demikian, JDK 8 tetap dapat digunakan untuk program yang kompatibel dengannya.

2. Encapsulation. Mengapa atribut Character bersifat private? Apa risikonya jika public? Mengapa setLevel() menghitung ulang HP dan attack di dalam class, bukan di pemanggilnya?

   Jawaban:

   Encapsulation adalah konsep dalam pemrograman berorientasi objek yang berfungsi untuk melindungi data di dalam sebuah class. Pada class Character dalam proyek Character Manager, atribut seperti id, name, type, level, hitPoints, attackPower, dan mana dibuat menggunakan akses private agar tidak dapat diakses atau diubah secara langsung dari luar class.

   Atribut dibuat private supaya data karakter tetap aman dan perubahan nilainya dapat dikontrol melalui method seperti getter dan setter. Sebagai contoh, method setMana(int mana) dapat memeriksa apakah nilai mana negatif. Jika nilainya negatif, program akan melempar IllegalArgumentException. Dengan demikian, nilai mana tetap mengikuti aturan yang telah ditentukan.

   Jika atribut dibuat public, class lain dapat mengubah nilainya secara langsung tanpa melalui validasi. Contohnya, atribut level dapat diubah menjadi -5 atau mana diisi dengan -100. Hal tersebut dapat menyebabkan data karakter menjadi tidak valid dan menimbulkan kesalahan dalam program.

   Selain itu, method setLevel() bertugas mengubah level karakter sekaligus menghitung ulang nilai hitPoints (HP) dan attackPower berdasarkan level terbaru. Proses perhitungan ini dilakukan di dalam class Character, bukan di class pemanggil, agar aturan perhitungan terpusat dan konsisten.

   Sebagai contoh, jika setiap class yang memanggil setLevel() harus menghitung ulang HP dan attack secara manual, ada kemungkinan salah satu class lupa memperbarui nilainya. Akibatnya, level karakter mungkin sudah berubah, tetapi HP dan attack masih menggunakan nilai lama. Dengan menempatkan perhitungan di dalam class Character, setiap perubahan level dapat dikelola secara terpusat sehingga mengurangi risiko kesalahan dan pengulangan kode.

3. Dua constructor dan enum. Kapan masing-masing constructor dipakai? Mengapa CharacterType berupa enum, bukan String? Berikan satu contoh bug jika memakai String.

   Jawaban:

   Dalam proyek Character Manager, class Character menggunakan dua constructor untuk memenuhi kebutuhan pembuatan objek yang berbeda.

   Pertama, constructor untuk membuat karakter baru. Constructor ini menerima parameter name, type, dan level. Constructor digunakan ketika pengguna ingin membuat karakter

baru. Berdasarkan parameter tersebut, program dapat menentukan data awal karakter, termasuk nilai hitPoints (HP) dan attackPower.

Contoh penggunaannya:

```
Character c1 = new Character("Ali", CharacterType.WARRIOR, 5);
```

Kode tersebut membuat karakter bernama Ali dengan tipe WARRIOR dan level 5.
Kedua, constructor untuk memuat data karakter yang sudah ada. Constructor ini menerima data yang lebih lengkap, seperti id, name, type, level, hitPoints, dan attackPower. Constructor tersebut dapat digunakan ketika data karakter dibaca kembali dari database sehingga informasi yang tersimpan dapat dipulihkan sesuai nilainya.

Contoh penggunaannya:

```
Character c2 = new Character(
    1, "Budi", CharacterType.MAGE, 3, 130, 35
);
```

Contoh ini berlaku jika constructor kedua dalam kode Anda memiliki urutan parameter tersebut.
Selanjutnya, CharacterType menggunakan enum, yaitu tipe data yang berisi sekumpulan konstanta yang telah ditentukan. Dalam proyek ini, enum memiliki empat jenis karakter, yaitu WARRIOR, MAGE, ARCHER, dan HEALER.
Enum dipilih daripada String karena membatasi pilihan tipe karakter pada nilai yang valid dan membantu compiler mendeteksi kesalahan penulisan. Selain itu, enum membuat kode lebih konsisten dan mudah dibaca.
Jika menggunakan String, kesalahan penulisan dapat menyebabkan bug.

Contohnya:

```
String tipe1 = "WARRIOR";
String tipe2 = "WARROIR";
```

Pada contoh tersebut, "WARROIR" merupakan kesalahan penulisan dari "WARRIOR", tetapi Java tetap menerima keduanya sebagai teks yang valid. Akibatnya, program mungkin tidak mengenali tipe karakter tersebut ketika melakukan pencarian atau percabangan berdasarkan jenis karakter.
Jika menggunakan enum, penulisannya menjadi:
CharacterType tipe = CharacterType.WARRIOR;
Apabila programmer menulis CharacterType.WARROIR, compiler akan memberikan kesalahan karena konstanta tersebut tidak tersedia dalam enum.

4. Diagram class (UML sederhana). Untuk class Character versi Anda (dengan mana dan isAlive()), tulis dalam format nama : tipe (akses) untuk atribut dan nama() : return type (akses) untuk method. Tabel diperbolehkan.

   Jawaban:

   Diagram Class UML Sederhana — Class Character

   1. Atribut

Nama	Tipe	Akses
id	int	private (-)
name	String	private (-)
type	CharacterType	private (-)
level	int	private (-)
hitpoint	int	private (-)
attackPower	int	private (-)
mana	int	private (-)

   2. Method

Nama Method	Return Type	Akses
Character()	Constructor	Public (+)
getId()	int	public (+)
getName()	String	public (+)
getType()	CharacterType	public (+)
getLevel()	int	public (+)
getHitPoints()	int	public (+)
getAttackPower()	int	public (+)
getMana()	int	public (+)
setName(String name)	void	public (+)
setType(CharacterType type)	void	public (+)
setLevel(int level)	void	public (+)
setHitPoints(int hitPoints)	void	public (+)
setAttackPower(int attackPower)	void	public (+)
setMana(int mana)	void	public (+)
isAlive()	boolean	public (+)
toString()	String	public (+)

3. Diagram UML

```
----------------------------------------
            Character
----------------------------------------
- id                :int
- name              :String
- type              :CharacterType
- level             :int
- hitPoints         :int
- attackPower       :int
- mana              :int
----------------------------------------
+ Character(...)
+ getId()                      :int
+ getName()                    :String
+ getType()                    :CharacterType
+ getLevel()                   :int
+ getHitPoints()               :int
+ getAttackPower()             :int
+ getMana()                    :int
+ setName(name)                :void
+ setType(type)                :void
+ setLevel(level)              :void
+ setHitPoints(hitPoints)      :void
+ setAttackPower(attackPower) : void
+ setMana(mana)                :void
+ isAlive()                    :boolean
+ toString()                   :String
----------------------------------------
```

Keterangan simbol UML:

- - berarti private, atribut hanya dapat diakses langsung dari dalam class.
- + berarti public, method dapat dipanggil dari luar class.
- boolean adalah tipe data yang menghasilkan nilai true atau false.

5. Refleksi. Jika harus menambah jenis karakter NECROMANCER, apa yang berubah di kode? Apakah desain sekarang sudah open for extension, closed for modification?

   Jawaban:

   Jika harus menambahkan jenis karakter baru bernama NECROMANCER pada proyek Character Manager, perubahan pertama yang dilakukan adalah menambahkan konstanta NECROMANCER ke dalam enum CharacterType.java.

Contoh kode sebelum perubahan:

```
public enum CharacterType {
   WARRIOR,
   MAGE,
   ARCHER,
   HEALER
}
```

Setelah ditambahkan jenis karakter baru:

```
public enum CharacterType {
   WARRIOR,
   MAGE,
   ARCHER,
   HEALER,
   NECROMANCER
}
```

Dengan perubahan tersebut, objek Character sudah dapat menggunakan tipe NECROMANCER.

Contohnya:

```
Character c = new Character(
   "Dark Mage",
   CharacterType.NECROMANCER,
   5
);
```

Jika NECROMANCER hanya menjadi jenis karakter tambahan tanpa kemampuan khusus, perubahan enum mungkin sudah cukup. Namun, jika karakter tersebut memiliki kemampuan unik, seperti memanggil makhluk bantuan, maka perlu ditambahkan logika atau class khusus untuk menangani perilakunya. Bagian program lain, seperti tampilan, penyimpanan database, dan pengujian, juga perlu diperiksa agar mendukung jenis karakter baru tersebut.

Desain Character Manager saat ini sudah menerapkan beberapa prinsip pemrograman berorientasi objek, seperti encapsulation, karena atribut dibuat private dan perubahan data dilakukan melalui method. Namun, desain ini belum sepenuhnya memenuhi prinsip Open/Closed Principle (OCP).

OCP berarti sebuah class harus terbuka untuk pengembangan fitur baru (open for extension), tetapi tertutup untuk perubahan kode yang sudah ada (closed for modification). Pada desain saat ini, penambahan NECROMANCER tetap mengharuskan perubahan pada enum CharacterType. Jika logika program menggunakan banyak percabangan berdasarkan tipe karakter, penambahan jenis baru juga dapat mengharuskan perubahan pada beberapa bagian kode.

Untuk meningkatkan penerapan OCP,dapat menggunakan interface atau pewarisan agar setiap jenis karakter memiliki perilaku khusus melalui class tersendiri. Dengan demikian, kemampuan baru dapat ditambahkan tanpa harus terus mengubah logika utama program.